NoSQL Assignment 2

IMT2022021 Rutul Patel
IMT2022050 Tanish Pathania
IMT2022076 Mohit Naik
IMT2022086 Ananthakrishna K

April 2026

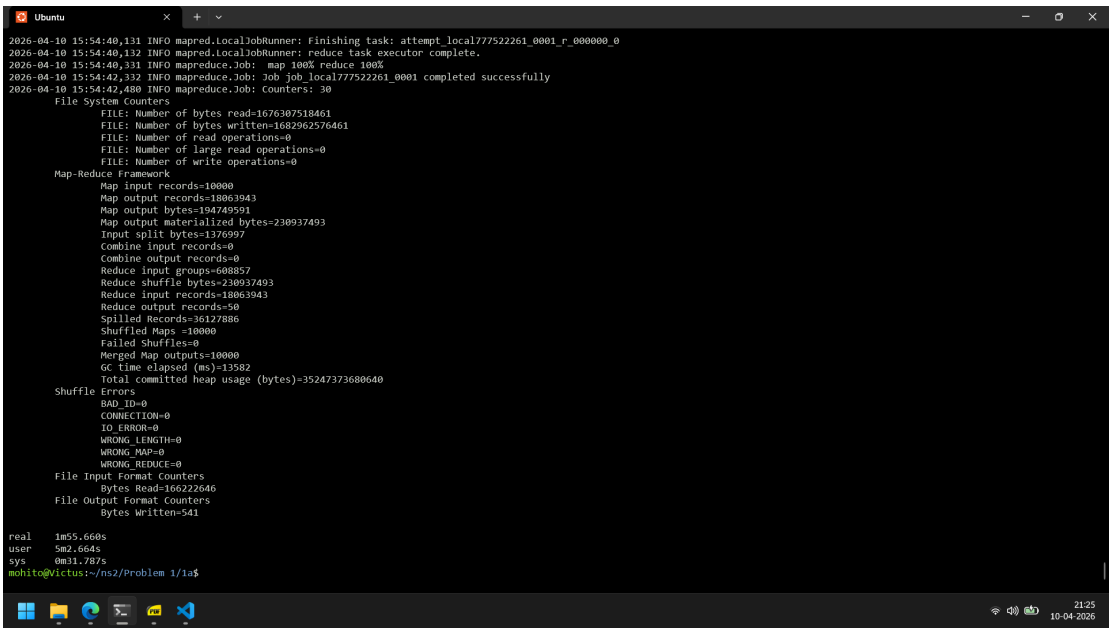
Part 1: Co-Occurring Word Matrix Generation: Pairs, Stripes and Local Aggregation

1a

The task was to identify the top 50 most frequently occurring words from a large Wikipedia dump while excluding common stop words. This was implemented using a MapReduce program following the standard word count pattern with additional filtering and ranking logic.

The program consists of a Mapper and a Reducer. In the Mapper phase, each input line is converted to lowercase and tokenized using a regular expression that removes non-word characters. Before emitting any word, the Mapper filters out stop words. These stop words are loaded during the setup phase using Hadoop's distributed cache mechanism, which ensures that the stopwords file is locally available to each mapper node. Only valid, non-empty, non-stop words are emitted as key-value pairs of the form (word, 1).

In the Reduce phase, all values associated with the same word are aggregated to compute the total frequency of that word across the dataset. Instead of directly outputting all word counts, the reducer maintains a `TreeMap` to track only the top 50 words. The `TreeMap` stores word counts as keys and words as values, automatically keeping them sorted. If the size exceeds 50, the smallest entry is removed. Finally, during the cleanup phase, the top 50 words are emitted in descending order of frequency.



```
2026-04-10 15:54:40,131 INFO mapred.LocalJobRunner: Finishing task: attempt_local777522261_0001_r_000000_0
2026-04-10 15:54:40,132 INFO mapred.LocalJobRunner: reduce task executor complete.
2026-04-10 15:54:40,331 INFO mapreduce.Job:  map 100% reduce 100%
2026-04-10 15:54:42,332 INFO mapreduce.Job: Job job_local777522261_0001 completed successfully
2026-04-10 15:54:42,480 INFO mapreduce.Job: Counters: 30
        File System Counters
                FILE: Number of bytes read=1676307518461
                FILE: Number of bytes written=1682962576461
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
        Map-Reduce Framework
                Map input records=10000
                Map output records=18063943
                Map output bytes=194749591
                Map output materialized bytes=230937493
                Input split bytes=1376997
                Combine input records=0
                Combine output records=0
                Reduce input groups=608857
                Reduce shuffle bytes=230937493
                Reduce input records=18063943
                Reduce output records=50
                Spilled Records=36127886
                Shuffled Maps =10000
                Failed Shuffles=0
                Merged Map outputs=10000
                GC time elapsed (ms)=13582
                Total committed heap usage (bytes)=35247373680640
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=166222646
        File Output Format Counters
                Bytes Written=541

real    1m55.660s
user    5m2.664s
sys     0m11.787s
mohito@Victus:~/ns2/Problem 1/1a$
```

Figure 1: Execution time for Top 50 Words (Part 1a)

1b

The co-occurrence matrix was constructed using the **pairs approach**, restricted to only the top 50 most frequent words obtained in part (1a). The objective is to compute how frequently pairs of these words occur within a fixed window distance d in the corpus.

In the Mapper phase, each input line is tokenized into lowercase words using a regular expression. During the setup phase, the list of top 50 words is loaded into memory using Hadoop's distributed cache (from `top50.txt`). This ensures that only relevant words are considered during processing.

For each word w_1 in the token sequence, if it belongs to the top 50 set, the mapper examines the next d words in the sequence. For each neighboring word w_2 within this window:

- If w_2 is also in the top 50 set and $w_1 \neq w_2$, a co-occurring pair is generated.
- The mapper emits both $(w_1, w_2) \to 1$ and $(w_2, w_1) \to 1$ to ensure symmetry in the co-occurrence matrix.

In the Reduce phase, all values corresponding to the same word pair are summed to compute the total co-occurrence count. The final output represents the co-occurrence matrix as word pairs with their corresponding frequencies.

d	Real Time
1	1m 41s
2	1m 44s
3	1m 47s
4	1m 58s

Table 1: Runtime of pairs approach for different window sizes

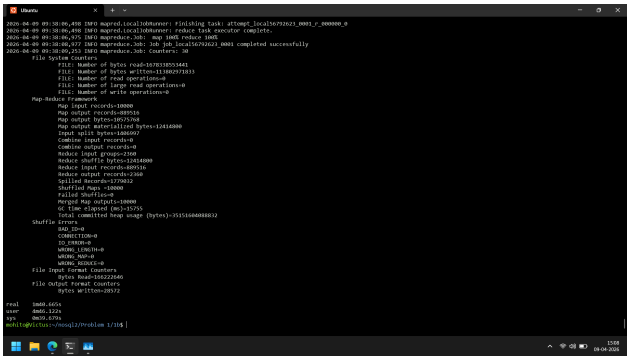


Figure 2: Pairs $d = 1$

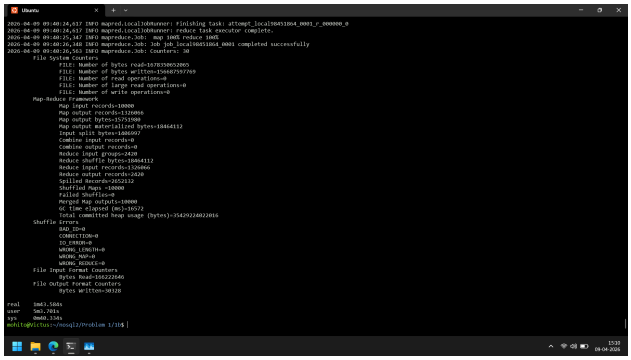


Figure 3: Pairs $d = 2$

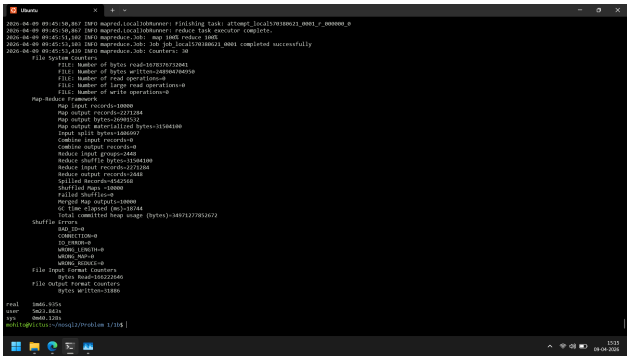


Figure 4: Pairs $d = 3$

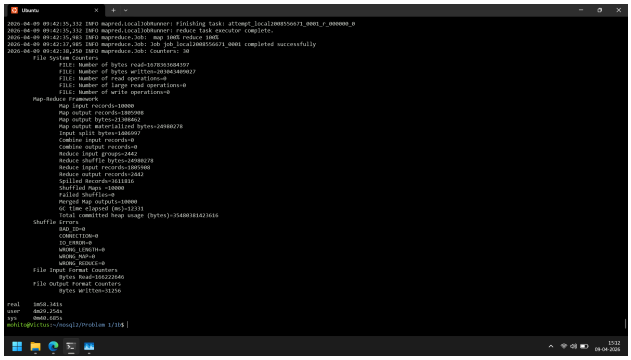


Figure 5: Pairs $d = 4$

1c

The co-occurrence matrix was constructed using the **stripes approach**, again restricted to the top 50 most frequent words obtained in part (1a). Unlike the pairs approach, which emits individual word pairs, the stripes

approach builds an associative array (or map) of neighboring word counts for each word.

Pseudocode for Stripes Mapper:

```
class Mapper:
    method setup(context):
        top50Words = load_cache("top50.txt")
        d = get_configuration("distance.d")

    method map(docId, text):
        tokens = tokenize(text)
        for i from 0 to length(tokens):
            w1 = tokens[i]
            if w1 not in top50Words: continue

            stripe = new Map()
            for j from (i - d) to (i + d):
                if i == j: continue
                w2 = tokens[j]
                if w2 in top50Words:
                    stripe[w2] = stripe.getOrDefault(w2, 0) + 1

            if stripe is not empty:
                emit(w1, stripe)
```

In the Reduce phase, all stripes corresponding to the same key w_1 are merged by summing the counts of each neighbor. This produces the final co-occurrence distribution for each word. The result is emitted as a formatted map representing the row of the co-occurrence matrix.

d	Real Time
1	1m 40s
2	1m 41s
3	1m 38s
4	1m 42s

Table 2: Runtime of stripes approach for different window sizes

1e

To fully evaluate performance optimizations, local aggregation (also known as the In-Mapper Combining pattern) was implemented and compared across two different architectural levels: **Map-Class level** and **Map-Function level**. These were applied to both the Pairs and Stripes approaches to minimize the amount of intermediate data transferred across the network during the shuffle and sort phase.

Map-Function Level Aggregation: In this approach, a local `HashMap` (or `MapWritable` for Stripes) is initialized entirely within the `map()` method. It aggregates the co-occurrences locally for a single input record (document line) and emits the partially aggregated results at the end of that specific `map()` call.

Map-Class Level Aggregation: In this approach, the aggregation structure is promoted to a class-level instance variable. It is initialized once per JVM task in the `setup()` method. As the mapper processes thousands of lines, it continually updates this global map instead of emitting intermediate values. Once all records for the split are processed, the mapper emits the fully aggregated map during the `cleanup()` method. This results in the absolute minimum amount of key-value pairs being emitted to the Reducer.

Pseudocode for Map-Class Pairs Aggregation:

```
class Mapper:
    method setup(context):
        localMap = new Map()
```

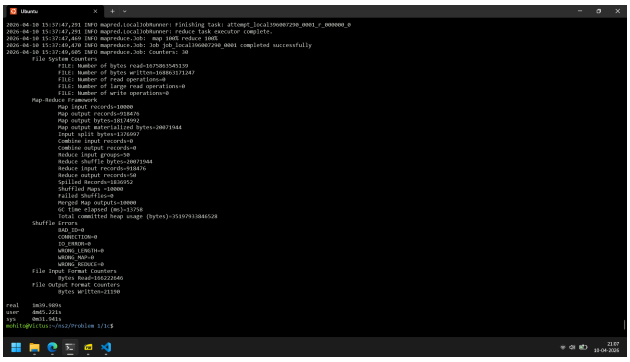


Figure 6: Stripes $d = 1$

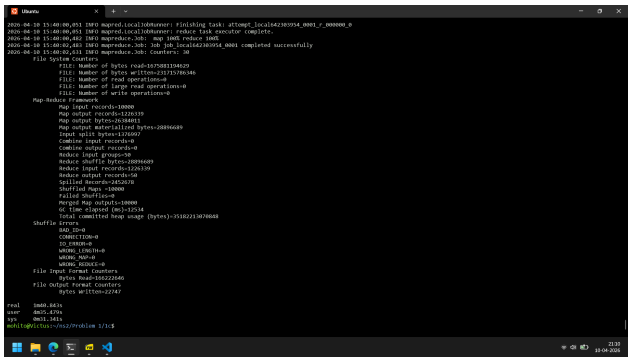


Figure 7: Stripes $d = 2$

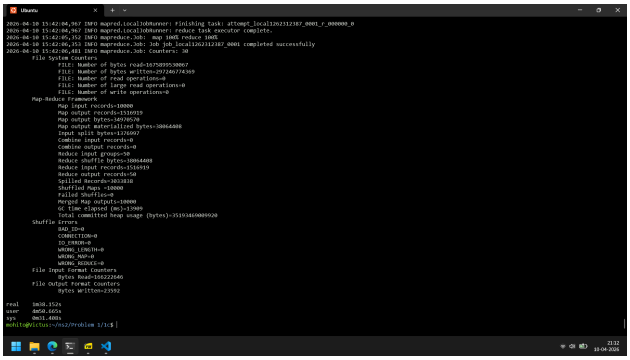


Figure 8: Stripes $d = 3$

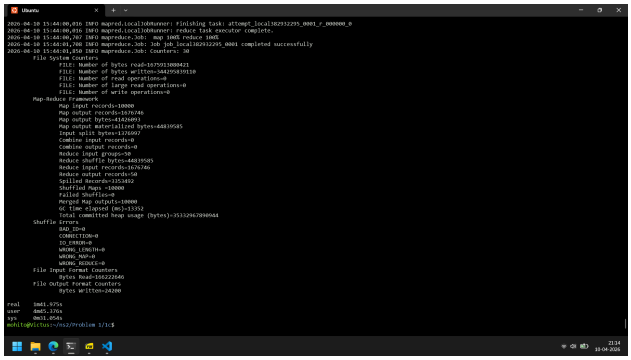


Figure 9: Stripes $d = 4$

```
method map(docId, text):
    // logic to identify co-occurring pairs w1 and w2
    pair1 = (w1, w2)
    pair2 = (w2, w1)

    // Accumulate locally instead of emitting
    localMap[pair1] = localMap.getOrDefault(pair1, 0) + 1
    localMap[pair2] = localMap.getOrDefault(pair2, 0) + 1

method cleanup(context):
    // Emit only once at the end of the Map task
    for pair, count in localMap:
        emit(pair, count)
```

d	Pairs (Map-Class)
1	1m 36s
2	1m 38s
3	1m 36s
4	1m 32s

Table 3: Runtime for Pairs (Map-Class) Across Different d Values

Part 2: Indexing Documents via Hadoop

2a: Document Frequency (DF)

The objective of this task was to compute the Document Frequency (DF) for each distinct term across the Wikipedia corpus. Unlike standard word count, DF measures the number of *distinct documents* a term appears in, regardless of how many times it repeats within a single document.

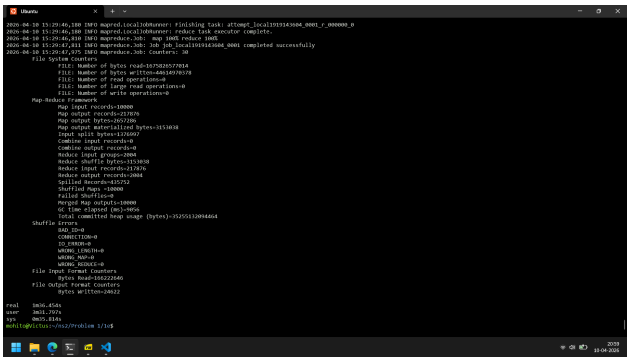


Figure 10: Map-Class Aggregation $d = 1$

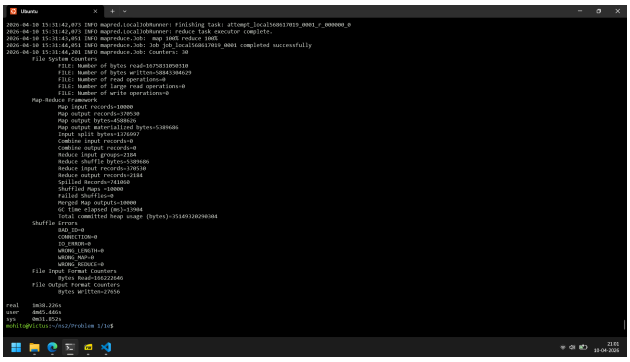


Figure 11: Map-Class Aggregation $d = 2$

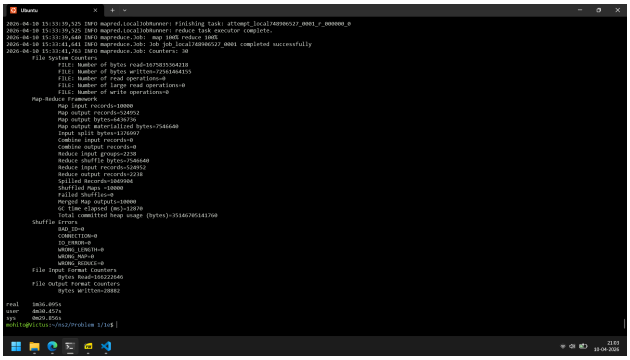


Figure 12: Map-Class Aggregation $d = 3$

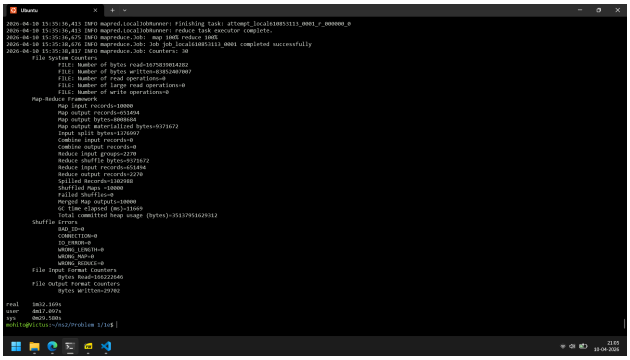


Figure 13: Map-Class Aggregation $d = 4$

In the **Mapper phase**, the program identifies the document being processed by retrieving the file name using the `FileSplit` context. Each line is tokenized, and the `stopwords.txt` file (loaded via Hadoop's Distributed Cache) is used to discard common terms. The remaining tokens are then reduced to their root forms using the `PorterStemmer` class from the `opennlp.tools.stemmer` library. The Mapper emits key-value pairs where the key is the `stemmed_term` and the value is the `docId`.

In the **Reducer phase**, the program receives a term along with an iterable list of document IDs. To ensure that multiple occurrences within the same document are only counted once, the Reducer utilizes a Java `HashSet`.

Pseudocode for DF Reducer:

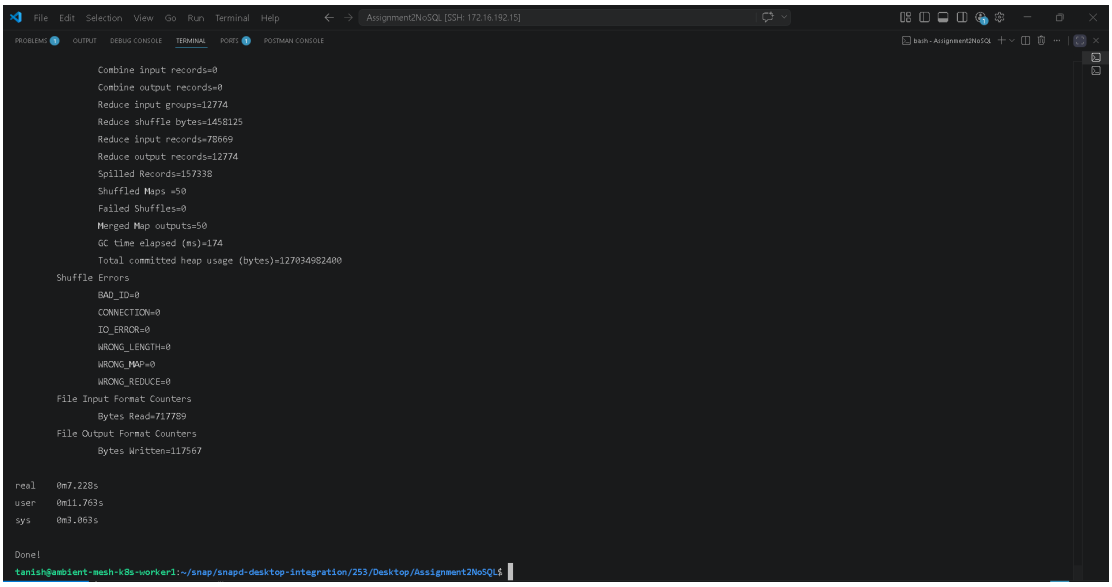
```
class Reducer:
    method reduce(term, docIds):
        uniqueDocs = new Set()

        for docId in docIds:
            uniqueDocs.add(docId) // Automatically filters duplicates

        emit(term, uniqueDocs.size())
```

Finally, an automated bash script isolates the output and extracts the top 100 terms with the highest DF into a `top100_df.tsv` file to be used in Part 2b.

2b: Term Frequency (TF) and Scoring

The objective of the final task was to compute a composite score for each document-term pair by combining the Term Frequency (TF) with the Document Frequency (DF) obtained in 2a.

In the **Mapper phase**, the program employs the Stripes algorithm to compute the local TF. During the `setup()` method, the `top100_df.tsv` file is loaded via Distributed Cache into a main-memory `HashMap`. As the Mapper tokenizes and stems the document text, it verifies if the stemmed token exists in the top 100 list. If it does, the term's count is incremented inside a local `MapWritable` stripe. Once the document is processed, the Mapper



```
File Edit Selection View Go Run Terminal Help
Assignment2NoSQL [SSH: 172.16.192.15]
PROBLEMS 1  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS 1  POSTMAN CONSOLE
bash - Assignment2NoSQL

                Combine input records=0
                Combine output records=0
                Reduce input groups=12774
                Reduce shuffle bytes=1458125
                Reduce input records=78669
                Reduce output records=12774
                Spilled Records=157338
                Shuffled Maps =50
                Failed Shuffles=0
                Merged Map outputs=50
                GC time elapsed (ms)=174
                Total committed heap usage (bytes)=127034982400
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=717789
        File Output Format Counters
                Bytes Written=117567

real    0m7.228s
user    0m11.763s
sys     0m3.063s

Done!
tanish@ambient-mesh-k8s-worker1:~/snap/snapd-desktop-integration/253/Desktop/Assignment2NoSQL$
```

Figure 14: Execution time for Document Frequency calculation (Part 2a)

```
Top 20 Results:
107301.txt      through 2.4962
107301.txt      44      2.6208
107301.txt      featur  2.6031
107301.txt      pt      2.4700
107301.txt      year    19.3749
107301.txt      american        5.0777
107301.txt      extern  2.4219
107301.txt      type    2.6031
107301.txt      major   2.4962
107301.txt      www     4.9400
107301.txt      commun  2.5862
107301.txt      state   22.5898
107301.txt      id      2.5242
107301.txt      titl    2.5242
107301.txt      us      2.3786
107301.txt      area    12.8490
107301.txt      11      2.5100
107301.txt      12      10.0967
107301.txt      apo     7.0177
107301.txt      14      5.1081

Done!
tanish@ambient-mesh-k8s-worker1:~/snap/snapd-desktop-integration/253/Desktop/Assignment2NoSQL$
```

Figure 15: Top 20 Document-Term Results generated by the TF Scoring script

emits the `docId` as the key and the entire TF stripe as the value.

In the **Reducer phase**, the program aggregates any partial stripes belonging to the same `docId` into a single, comprehensive `HashMap` representing the complete Term Frequency distribution for that document. The Reducer then iterates through the map and calculates the final score for each term using the formula provided in the assignment:

$$SCORE = TF \times \log_{10}\left(\frac{10000}{DF} + 1\right)$$

The final data is emitted in the required schema: `ID<tab>TERM<tab>SCORE`.